

Fundamentos de Python

**Autor:**

Romulo Salvador

Semillero de Investigación en Business Analytics (SYBA UNALM)

GitHub: <https://github.com/Romulo-Salvador>

LinkedIn: <https://www.linkedin.com/in/romulo-salvador/>

ORCID: <https://orcid.org/0009-0000-7998-1611>

Índice

1. Introducción a Python

Usos y aplicaciones de Python. Áreas de desarrollo destacadas. Entornos de trabajo (PyCharm, Jupyter, Azure, Colab)

2. Google Colab

Atajos básicos. Tipos de celdas: Código y Markdown

3. Tipos de datos y variables

3.1. Numéricos (int, float)

3.2. Booleanos

3.3. Cadenas de texto (strings Y entrada de usuario)

4. Operadores

4.1. Aritméticos

4.2. Comparación (orden lexicográfico) 4.3. Lógicos

4.4. Asignación

4.5. Identidad

4.6. Pertenencia

5. Estructuras de control

5.1. Condicionales: if, elif, else

5.2. Bucles: for, while

6. Estructuras de datos

6.1. Listas

6.2. Matrices

6.3. Cadenas de caracteres

6.4. Tuplas

6.5. Conjuntos

6.6. Diccionarios

7. Funciones y control de flujo

7.1. break

7.2. continue

7.3. range

7.4. pass

7.5. Definición de funciones (def, return, parámetros por defecto)

7.6. Funciones lambda y composición

8. Clases y objetos

8.1. Definición de clases

8.2. Creación de objetos

8.3. Métodos de objetos

1. Introducción a Python

Python se destaca por su versatilidad y amplia comunidad, lo que permite crear aplicaciones robustas para backend, automatización y análisis avanzado. Además, es muy usado en ciencia de datos, inteligencia artificial, ciberseguridad y desarrollo de herramientas multiplataforma.

Desarrollos interesantes en el mundo de Python:

- **PyPI:** El sistema de paquetes de Python, PyPI, ha superado los 200 millones de descargas al mes, lo que lo convierte en uno de los repositorios de paquetes más populares en el mundo.
- **Aplicaciones en la nube:** Python está ganando terreno como lenguaje de elección para la implementación de aplicaciones en la nube, gracias a su amplia gama de bibliotecas y herramientas de desarrollo.
- **Científico de datos:** Python sigue siendo un lenguaje popular para la ciencia de datos, con una creciente comunidad de usuarios que utilizan bibliotecas como NumPy, Pandas

y Matplotlib para analizar y visualizar grandes conjuntos de datos.

- Automatización: Python también es ampliamente utilizado en la automatización de tareas, especialmente en áreas como la automatización de pruebas, la gestión de redes y la automatización de trabajos en segundo plano.

Entornos de trabajo para Python:

- PyCharm
- Jupyter Notebooks
- Azure Notebooks
- Google Collaboratory

2. Introducción a Colab

Atajos:

`Ctrl + m + a` Crear una celda sobre la actual

`Ctrl + m + b` Crear una celda debajo de la actual

`Ctrl + m + d` Borrar celda

2.1. Tipos de celdas

Para la ejecución de código en celdas se puede usar el botón play o la combinación de teclas `Shift + Enter`

```
In [ ]: print("Esto es una celda de código")
```

Esto es una celda de código

```
In [ ]: #Esto es un comentario en una celda de código
5 + 4
```

```
Out[ ]: 9
```

Las siguiente son inputs y outputs en celdas Markdown

Las siguiente son inputs y outputs en celdas Markdown

Esto es un título 1

Esto es un título 1

Esto es un título 2

Esto es un título 2

Esto es un título 3

Esto es un título 3

Esto es un texto en cursiva

Esto es un texto en cursiva

****Esto es un texto en negrita****

Esto es un texto en negrita

Esto es una lista:

1. Elemento de la lista
2. Elemento de la lista

Esto es una lista:

1. Elemento de la lista
2. Elemento de la lista

3. Tipos de datos y variables

A diferencia de otros lenguajes de programación, Python no requiere que se declaren los tipos de datos

3.1. Numéricas

```
In [ ]: 32
```

```
Out[ ]: 32
```

```
In [ ]: # La ejecución solo muestra el último comando
x=32
print(x) #Si usamos comandos print se muestran varias filas de resultados
x
```

```
32
```

```
Out[ ]: 32
```

```
In [ ]: #tipo de variable
type(x)
```

```
Out[ ]: int
```

```
In [ ]: y = 45.0
type(y)
```

```
Out[ ]: float
```

```
In [ ]: z = int(y)
print(z)
type(z)
```

```
45
```

```
Out[ ]: int
```

```
In [ ]: y = 45.9  
int(y) # solo toma en cuenta la parte entera, función SUELO
```

```
Out[ ]: 45
```

```
In [ ]: #Impresion de varios valores en una sola linea de codigo  
x, y, z
```

```
Out[ ]: (32, 45.9, 45)
```

3.2. Booleanos

```
In [ ]: a = True  
b = False  
print(a)  
b
```

```
Out[ ]: True  
False
```

```
In [ ]: int(a)
```

```
Out[ ]: 1
```

```
In [ ]: float(b)
```

```
Out[ ]: 0.0
```

3.3. Cadenas

```
In [ ]: cadena = "Hola, mundo!"  
cadena
```

```
Out[ ]: 'Hola, mundo!'
```

```
In [ ]: type(cadena)
```

```
Out[ ]: str
```

```
In [ ]: 'Hola, "Alejandro" es mi nombre'
```

```
Out[ ]: 'Hola, "Alejandro" es mi nombre'
```

```
In [ ]: "Hola, 'Manuel' es mi nombre"
```

```
Out[ ]: "Hola, 'Manuel' es mi nombre"
```

Entrada de valores por usuario

```
In [ ]: nombre = input()
```

```
Romulo
```

```
In [ ]: nombre*5
```

```
Out[ ]: 'RomuloRomuloRomuloRomuloRomulo'
```

```
In [ ]: saludo = "Hola, " + nombre
saludo
```

```
Out[ ]: 'Hola, Romulo'
```

```
In [ ]: "python" + str(3)
```

```
Out[ ]: 'python3'
```

```
In [ ]: saludo.lower()
```

```
Out[ ]: 'hola, romulo'
```

```
In [ ]: saludo.upper()
```

```
Out[ ]: 'HOLA, ROMULO'
```

```
In [ ]: saludo = saludo.upper()
saludo
```

```
Out[ ]: 'HOLA, ROMULO'
```

```
In [ ]: "Natalia Malaga" == "natalia malaga"
```

```
Out[ ]: False
```

4. Operadores

4.1. Operadores Aritméticos

Adición

```
In [ ]: # ¿Cuánto dura un partido de fútbol con tiempo extra?
45 + 45 + 15 + 15
```

```
Out[ ]: 120
```

Multiplicación

```
In [ ]: # ¿Cuántos segundos tiene un día?
segundos = 24 * 60 * 60
segundos
```

```
Out[ ]: 86400
```

División

```
In [ ]: 47 / 9
```

```
Out[ ]: 5.222222222222222
```

División entera

```
In [ ]: int(47/9)
```

Out[]: 5

In []: 47 // 9

Out[]: 5

Residuo de una división

In []: 47 % 9

Out[]: 2

Potencia

In []: 3 ** 2

Out[]: 9

Raíz cuadrada

In []: 16 ** (1/2)

Out[]: 4.0

Paréntesis para orden de operaciones

In []: 4 + 5 * 6

Out[]: 34

In []: (4 + 5) * 6

Out[]: 54

In []: 4 < 5

Out[]: True

4.2. Operadores de Comparación

Igualdad

In []: 8==(2*4)

Out[]: True

Desigualdad

In []: 8!=5

Out[]: True

Menor

In []: 3<4

Out[]: True

Menor o igual que

In []: `4<=4`

Out[]: True

In []: `3<=5`

Out[]: True

Mayor

In []: `8>4`

Out[]: True

Mayor o igual que

In []: `4>=4`

Out[]: True

In []: `6>=3`

Out[]: True

El orden lexicográfico

Es el orden en el que se colocan las palabras en un diccionario. En Python, el orden lexicográfico se aplica a los objetos de tipo string, que se comparan carácter por carácter. Si un carácter en una cadena es menor que otro en otra cadena, la primera cadena es considerada menor que la segunda.

```
In [ ]: print("apple" < "banana")
        print("cat" < "dog")
        print("egg" < "eggs")

        print("Zebra" < "apple")
        print("apple" < "Apple")
```

True
True
True
True
False

El orden lexicográfico en Python es sensible a mayúsculas y minúsculas, por lo que las cadenas que contienen letras mayúsculas se colocan antes que las que contienen solo letras minúsculas. Es importante tener en cuenta que en Python, el orden lexicográfico se aplica a todos los tipos de secuencias, incluyendo listas, tuplas y conjuntos, y no solo a las cadenas.

4.3. Operadores Lógicos


```
In [ ]: a=True  
        b=False
```

And

```
In [ ]: a and a
```

```
Out[ ]: True
```

```
In [ ]: a and b
```

```
Out[ ]: False
```

Or

```
In [ ]: a or b
```

```
Out[ ]: True
```

```
In [ ]: # xor  
        a ^ b
```

```
Out[ ]: True
```

Not

```
In [ ]: not a
```

```
Out[ ]: False
```

Lógica proposicional

```
In [ ]: not((True and False) or (False and True))
```

```
Out[ ]: True
```

4.4. Operadores de Asignación

Asignación simple

```
In [ ]: x = 10  
        x
```

```
Out[ ]: 10
```

Asignación múltiple

```
In [ ]: x = y = z = 10  
        x, y, z
```

```
Out[ ]: (10, 10, 10)
```

Adición de asignación

```
In [ ]: x += 10
x
```

Out[]: 20

Sustracción de asignación

```
In [ ]: x -= 10
x
```

Out[]: 10

Multiplicación de asignación

```
In [ ]: x *= 10
x
```

Out[]: 100

División de asignación

```
In [ ]: x /= 10
x
```

Out[]: 10.0

Módulo de asignación

```
In [ ]: x %= 10
x
```

Out[]: 0.0

Potencia de asignación

```
In [ ]: x **= 10
x
```

Out[]: 0.0

División entera de asignación

```
In [ ]: x //= 10
x
```

Out[]: 0.0

Además, en Python también es posible utilizar la asignación condicional con el operador ternario: *if-else*

```
In [ ]: x = 5 if y > 5 else 20
x
```

Out[]: 5

4.5. Operaciones de Identidad

Is

```
In [ ]: x = [1, 2, 3]
        y = x
        x is y
```

```
Out[ ]: True
```

```
In [ ]: x = [1, 2, 3]
        y = [1, 2, 3]
        x is not y
```

```
Out[ ]: True
```

Es importante tener en cuenta que los operadores de identidad se utilizan para comparar objetos en memoria y no valores. Por lo tanto, dos objetos que contienen los mismos valores pueden ser objetos diferentes en memoria.

```
In [ ]: x = [1, 2, 3]
        y = [1, 2, 3]
        x == y, x is y
```

```
Out[ ]: (True, False)
```

4.6. Operadores de Pertenencia

In

```
In [ ]: x = [1, 2, 3]
        2 in x
```

```
Out[ ]: True
```

```
In [ ]: 4 not in x
```

```
Out[ ]: True
```

Estos operadores se pueden utilizar con diferentes tipos de objetos, como listas, tuplas, conjuntos, diccionarios, etc.

```
In [ ]: x = {'a': 1, 'b': 2}
        'a' in x
```

```
Out[ ]: True
```

5. Estructuras de Control

5.1. Condicionales

5.1.1. If

```
In [ ]: a = 33
        b = 200
        if b > a:
            print("b es mayor que a")
```

b es mayor que a

5.1.2. Elif

```
In [ ]: a = 33
        b = 33
        if b > a:
            print("b es mayor que a")
        elif a == b:
            print("a y b son iguales")
```

a y b son iguales

5.1.3. Else

```
In [ ]: a = 200
        b = 33
        if b > a:
            print("b es mayor que a")
        elif a == b:
            print("a y b son iguales")
        else:
            print("a es mayor que b")
```

a es mayor que b

5.2. Iterativas

5.2.1. For

```
In [ ]: fruits = ["apple", "banana", "cherry"]
        for x in fruits:
            print(x)
```

apple
banana
cherry

5.2.2. While

```
In [ ]: i = 1
        while i < 6:
            print(i) # mientras i es menor que 6, se imprime
            i += 1
```

1
2
3
4
5

6. Estructuras de Datos

6.1. Listas

```
In [ ]: lista=[3,2,1,3]
        lista
```

```
Out[ ]: [3, 2, 1, 3]
```

```
In [ ]: type(lista) # tipo de estructura
```

```
Out[ ]: list
```

```
In [ ]: len(lista) # Longitud o numero de elementos
```

```
Out[ ]: 4
```

```
In [ ]: lista[0] # llamar un elemento
```

```
Out[ ]: 3
```

```
In [ ]: lista[1:] # llamar un intervalo[cerrado:abierto]
```

```
Out[ ]: [2, 1, 3]
```

```
In [ ]: lista.sort() # ordenar la lista
        lista
```

```
Out[ ]: [1, 2, 3, 3]
```

```
In [ ]: lista.sort(reverse=True) # ordenar la lista de forma descendente
        lista
```

```
Out[ ]: [3, 3, 2, 1]
```

```
In [ ]: lista.index(3) # imprime la posición de 3, solo el más cercano
```

```
Out[ ]: 0
```

```
In [ ]: lista.count(3) # imprime las veces que está repetido n
```

```
Out[ ]: 2
```

```
In [ ]: 2 in lista # imprime un booleano indicando si está en la lista
```

```
Out[ ]: True
```

```
In [ ]: lista.reverse() # invierte el orden de la lista
        lista
```

```
Out[ ]: [1, 2, 3, 3]
```

Modificar elementos de listas

```
In [ ]: c2 = [i**2 for i in lista] # números al cuadrado
        c2
```

```
Out[ ]: [1, 4, 9, 9]
```

```
In [ ]: lista[2] = "Scala"
        lista
```

```
Out[ ]: [1, 2, 'Scala', 3]
```

Aumentar elementos

```
In [ ]: lista*3 # triplica len(lista)
```

```
Out[ ]: [1, 2, 'Scala', 3, 1, 2, 'Scala', 3, 1, 2, 'Scala', 3]
```

```
In [ ]: lista.append(3) # agregar al final, ID constante  
lista
```

```
Out[ ]: [1, 2, 'Scala', 3, 3]
```

```
In [ ]: lista = lista + [3] # ID diferente  
lista
```

```
Out[ ]: [1, 2, 'Scala', 3, 3, 3]
```

```
In [ ]: lista.extend([7,8,9]) # agregar varios  
lista
```

```
Out[ ]: [1, 2, 'Scala', 3, 3, 3, 7, 8, 9]
```

```
In [ ]: lista.insert(0,20) # insertar "(posicion, valor)"  
lista
```

```
Out[ ]: [20, 1, 2, 'Scala', 3, 3, 3, 7, 8, 9]
```

Eliminar elementos

```
In [ ]: del lista [-1]  
lista
```

```
Out[ ]: [20, 1, 2, 'Scala', 3, 3, 3, 7, 8]
```

```
In [ ]: lista.pop(0) # borrar la posocion 0, default -1
```

```
Out[ ]: 20
```

```
In [ ]: lista.remove(3) # elimina el valor indicado, solo una vez, la primera coincidencia  
lista
```

```
Out[ ]: [1, 2, 'Scala', 3, 3, 7, 8]
```

```
In [ ]: lista.clear() # elimina todos los valores de la lista  
lista
```

```
Out[ ]: []
```

```
In [ ]: lista1 = lista.copy() # diferente ID  
print(id(lista))  
id(lista1)
```

```
138915332708928  
Out[ ]: 138915332546112
```

```
In [ ]: lista2 = lista # igual ID  
print(id(lista))
```

```
id(lista2)
```

```
138915332708928
```

```
Out[ ]: 138915332708928
```

6.2. Matrices

```
In [ ]: matriz= [[1,2],[3,4]]  
matriz[0][1]
```

```
Out[ ]: 2
```

```
In [ ]: matriz[1:][0:] # (matriz[1:][:1]
```

```
Out[ ]: [[3, 4]]
```

```
In [ ]: [lista]*3 # crea una matriz, igual ID
```

```
Out[ ]: [[], [], []]
```

Otra forma de crear matrices:

```
In [ ]: notas=[]  
x=3  
for i in range(x): # x filas  
    notas.append([0]*x) # x columnas, ID diferente  
notas
```

```
Out[ ]: [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Entrada de valores de una matriz

```
In [ ]: f=int(input("ingrese el número de filas: "))  
c=int(input("ingrese el número de columnas: "))  
notas=[]  
for i in range(0,f):  
    notas.append([0]*c)  
print(notas)  
for i in range(f):  
    for j in range(c):  
        print("ingrese la nota del ",i+1,"-ésimo estudiantes y del ",j+1,"-ésimo cu  
            #nota =float(input())  
            notas[i][j] = float(input())  
print(notas)
```

```

ingrese el número de filas: 3
ingrese el número de columnas: 2
[[0, 0], [0, 0], [0, 0]]
ingrese la nota del 1 -ésimo estudiantes y del 1 -ésimo curso:
15
ingrese la nota del 1 -ésimo estudiantes y del 2 -ésimo curso:
13
ingrese la nota del 2 -ésimo estudiantes y del 1 -ésimo curso:
11
ingrese la nota del 2 -ésimo estudiantes y del 2 -ésimo curso:
17
ingrese la nota del 3 -ésimo estudiantes y del 1 -ésimo curso:
18
ingrese la nota del 3 -ésimo estudiantes y del 2 -ésimo curso:
12
[[15.0, 13.0], [11.0, 17.0], [18.0, 12.0]]

```

6.3. Cadena de Caracteres (inmutable)

```

In [ ]: # la cadena se comporta como una lista
cad= "hola mundo" # creando cadena
cad[0] # imprime la posición 0 de la cadena

```

```

Out[ ]: 'h'

```

```

In [ ]: cad[-1]

```

```

Out[ ]: 'o'

```

```

In [ ]: cad[2:9]

```

```

Out[ ]: 'la mund'

```

```

In [ ]: list(cad)

```

```

Out[ ]: ['h', 'o', 'l', 'a', ' ', 'm', 'u', 'n', 'd', 'o']

```

```

In [ ]: cad + 's' # concatenando

```

```

Out[ ]: 'hola mundos'

```

```

In [ ]: cad.replace('h','o') # reemplazamos 'h' por 'o'

```

```

Out[ ]: 'oola mundo'

```

```

In [ ]: cad.upper() # la convierte a mayúsculas

```

```

Out[ ]: 'HOLA MUNDO'

```

```

In [ ]: for elem in cad: # cad se comporta como un range
        print(elem)

```


h
o
l
a

m
u
n
d
o

```
In [ ]: # k toma los valores [longitud, -1>, sumando -1(default:+1)
        for k in range(len(cad)-1, -1, -1):
            print(cad[k])
```

o
d
n
u
m

a
l
o
h

6.4. Tuplas

```
In [ ]: #La tupla es como una lista pero que no puede ser editada
        tupla=(4,5,8,"contenido","elemento", [1,2,3],6.9) #creacion de tuplas
        #Los mismos codigos de lista funcionan con la tupla(*claro, cumplan la condicion de
        lista = list(tupla) # conviertes la tupla en una lista
        tupla = tuple(lista) #conviertes la lista en una tupla
```

6.5. Conjuntos

```
In [ ]: conjunto =set() #si no ponemos esto python lo toma como un diccionario
        conjunto = {1,2,"letter", 4.21} # al hacer esos dos pasos tenemos un conjunto
        #en un conjunto puede ir cualquier valor menos otras colecciones
        conjunto.add(5) #agrega el valor indicado en cualquier lugar del conjunto
        conjunto
```

```
Out[ ]: {1, 2, 4.21, 5, 'letter'}
```

```
In [ ]: conjunto.discard(1) #elimina el valor indicado del conjunto
        conjunto
```

```
Out[ ]: {2, 4.21, 5, 'letter'}
```

```
In [ ]: conjunto.clear() #borra todos los elementos del conjunto
        conjunto
```

```
Out[ ]: set()
```

```
In [ ]: 3 in conjunto # verifica si el valor indicado esta en el conjunto
```

```
Out[ ]: False
```

```
In [ ]: 3 not in conjunto #verifica si el valor indicado no esta en el conjunto
```

```
Out[ ]: True
```

Operaciones con conjuntos (dados los conjuntos a y b)

```
In [ ]: a = set()
a = {1,2,3,4,5}
b = set()
b = {4,5,6,7,8}
```

```
In [ ]: c = a | b # al hacer eso unes dos conjuntos
c
```

```
Out[ ]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [ ]: c = a & b # interseccion de conjuntos
c
```

```
Out[ ]: {4, 5}
```

```
In [ ]: c = a - b # todo a menos b
c
```

```
Out[ ]: {1, 2, 3}
```

```
In [ ]: c = a ^ b # diferencia simetrica
c
```

```
Out[ ]: {1, 2, 3, 6, 7, 8}
```

```
In [ ]: a.issubset(c) # verifica si a es un subconjunto de c
```

```
Out[ ]: False
```

```
In [ ]: c.issuperset(a) #verifica si c es un superconjunto de a
```

```
Out[ ]: False
```

```
In [ ]: a.isdisjoint(b) #verifica si a y b son disconjuntos
```

```
Out[ ]: False
```

```
In [ ]: a = frozenset({1,2,3}) #convierte immutable a un conjunto
```

6.6. Diccionarios

{index ó keys: values} = {índices ó llaves: valores}

```
In [ ]: # los valores de los significantes(values) pueden ser listas o tuplas, e incluso diccionarios
diction = {'a': 'armadillo', 'b': 'burro', 'c': 'cabra'}
diction['a'] # imprime el value del index 'a'
```

```
Out[ ]: 'armadillo'
```

```
In [ ]: diction.values() # imprime (armadillo, burro, ...)
```

```
Out[ ]: dict_values(['armadillo', 'burro', 'cabra'])
```

```
In [ ]: diction.keys() # imprime(a,b,c)
```

```
Out[ ]: dict_keys(['a', 'b', 'c'])
```

```
In [ ]: list(diction.values()).index('cabra') # busca el indice de la cadena "cabra"
```

```
Out[ ]: 2
```

```
In [ ]: list(diction.keys())[list(diction.values()).index('cabra')] # busca la llave de la
```

```
Out[ ]: 'c'
```

```
In [ ]: diction['d'] = 'delfin' # crea o edita un elemento  
diction
```

```
Out[ ]: {'a': 'armadillo', 'b': 'burro', 'c': 'cabra', 'd': 'delfin'}
```

```
In [ ]: # busca en index's y si no lo encuentra imprime: "no existe ese valor"  
diction.get("b", "no existe ese valor")
```

```
Out[ ]: 'burro'
```

Creación de tuplas dentro de una lista

```
In [ ]: diction.items()
```

```
Out[ ]: dict_items([('a', 'armadillo'), ('b', 'burro'), ('c', 'cabra'), ('d', 'delfin')])
```

```
In [ ]: len(diction) # cuenta cuantos index hay en el diccionario
```

```
Out[ ]: 4
```

```
In [ ]: 'a' in diction # verifica si un index esta presente
```

```
Out[ ]: True
```

```
In [ ]: 'armadillo' in diction.values() # si un value esta presente
```

```
Out[ ]: True
```

```
In [ ]: 'burro' == diction['b'] # true, mismo ID
```

```
Out[ ]: True
```

```
In [ ]: 'burro' in diction['b'] # true , mismo ID
```

```
Out[ ]: True
```

```
In [ ]: for animal in diction:  
        # print(animal) # a,b,c  
        # print(diction[animal]) # armadillo,burro,..  
        if diction[animal] == 'burro':  
            print(animal)
```

b

```
In [ ]: diction.clear() #borra todos lo elementos
```

```
diction
```

```
Out[ ]: {}
```

7. Funciones

7.1. Break (ruptura)

```
In [ ]: # Salga del bucle cuando sea "banana":x
fruits = ["apple", "banana", "cherry", "pear"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

```
apple
banana
```

```
In [ ]: # Salir del bucle cuando es "plátano", Pero esta vez el descanso llega antes de la
fruits = ["apple", "banana", "cherry", "pear"]
for x in fruits:
    if x == "banana":
        break
    print(x)
```

```
apple
```

7.2. Continue

```
In [ ]: # Con la declaración continue podemos detener la iteración actual del bucle y conti
fruits = ["apple", "banana", "cherry", "pear"]
for x in fruits:
    if x == "banana": # No imprime plátano
        continue
    print(x)
```

```
apple
cherry
pear
```

7.3. Range

```
In [ ]: # Devuelve una secuencia de números, comenzando desde 0 por defecto, e incrementos
for x in range(6):
    print(x)
```

```
0
1
2
3
4
5
```

```
In [ ]: # Incremente la secuencia con 3 (el valor predeterminado es 1):
for x in range(2, 20, 3):
    print(x)
```

2
5
8
11
14
17

7.4. Pass

```
In [ ]: # Los bucles no pueden estar vacíos, pero si para alguna razón tiene un bucle sin c
for x in [0, 1, 2]:
    pass
```

7.5. Def (funciones)

```
In [ ]: def sumar1(x,y): # x & y son los argumentos de entrada
        return(x+y)
resultado=sumar1(3,4)
print(resultado)
type(resultado)
```

7
int

Out[]:

Sin Return

```
In [ ]: def sumar1(x,y): # x & y son los argumentos de entrada
        print(x+y)
resultado=sumar1(3,4)
print(resultado)
type(resultado) # NoneType es un tipo de datos que representa la ausencia de valor
```

7
None
NoneType

Out[]:

Argumentos por default

```
In [ ]: def sumar(x,y=1): # y toma un valor por default
        suma=x+y
        return (suma,x,y)
sumar(3)
```

Out[]: (4, 3, 1)

Una función que lee una lista de elementos numéricos.

```
In [ ]: def crearLista(n): # n es argumento de entrada para el número de elementos en la Li
        L = []
        #n = int(input("ingrese el número de elementos: "))
        for e in range(n):
            print("ingrese el ",e+1," elemento:")
            elem = float(input())
            L.append(elem)
        return (L)
A = crearLista(3) # A toma el valor de la lista creada
A
```

```
ingrese el 1 elemento:
1
ingrese el 2 elemento:
2
ingrese el 3 elemento:
4
Out[ ]: [1.0, 2.0, 4.0]
```

```
In [ ]: crearLista(3)[0]
```

```
ingrese el 1 elemento:
5
ingrese el 2 elemento:
7
ingrese el 3 elemento:
9
Out[ ]: 5.0
```

De una lista retorna el valor de la media

```
In [ ]: def media(lista):
        n = len(lista)
        media = sum(lista)/n
        return media
        media([0,1,2,3,4])
```

```
Out[ ]: 2.0
```

Composicion de funciones

```
In [ ]: media(crearLista(4)) # f(g(x))
```

```
ingrese el 1 elemento:
2
ingrese el 2 elemento:
4
ingrese el 3 elemento:
6
ingrese el 4 elemento:
8
Out[ ]: 5.0
```

```
In [ ]: def aprobado(nota):
        return nota>=10.5
        aprobado(media(crearLista(4))) # h(f(g(x)))
```

```
ingrese el 1 elemento:
13
ingrese el 2 elemento:
17
ingrese el 3 elemento:
4
ingrese el 4 elemento:
5
Out[ ]: False
```

7.6. Lambda

```
In [ ]: x = lambda a : a + 10
        x(5)
```

```
Out[ ]: 15
```

```
In [ ]: # Pueden tomar cualquier número de argumentos
x = lambda a, b, c : a + b + c
x(5, 6, 2)
```

```
Out[ ]: 13
```

Función anónima durante un corto período de tiempo

```
In [ ]: def myfunc(n):
        return lambda a : a * n
```

```
In [ ]: mydoubler = myfunc(2) # Duplica el número
mydoubler(11)
```

```
Out[ ]: 22
```

```
In [ ]: mytripler = myfunc(3) # Triplica el número
mytripler(11)
```

```
Out[ ]: 33
```

8. Clases y objetos

8.1. Clases

```
In [ ]: class MyClass: # clase denominada MyClass
        x = 5          # con una propiedad denominada x
```

8.2. Objetos

```
In [ ]: p1 = MyClass() # objeto denominado p1
p1.x
```

```
Out[ ]: 5
```

La función ***init*** ()

```
In [ ]: class persona:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        p1 = persona("John", 36)

        print(p1.name)
        print(p1.age)
```

```
John
36
```

```
In [ ]: print(p1)
```

```
<__main__.persona object at 0x7e57be7538f0>
```

La función **str()**

```
In [ ]: class persona:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        def __str__(self):
            return f"{self.name}({self.age})"

p2 = persona("John", 36)
print(p2)
p2
```

John(36)

```
Out[ ]: <__main__.persona at 0x7e57be753980>
```

8.3. Métodos de Objetos

```
In [ ]: class persona:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        def myfunc(self):
            print("Hello my name is " + self.name)

p1 = persona("John", 36)
p1.myfunc()
```

Hello my name is John

El parámetro *self*

Es una referencia a la instancia actual de la clase, y se utiliza para acceder a las variables que pertenecen a la clase.*self*

```
In [ ]: class persona:
        def __init__(mysillyobject, name, age): # mysillyobject en lugar de self
            mysillyobject.name = name
            mysillyobject.age = age

        def myfunc(abc): # abc en lugar de self
            print("Hello my name is " + abc.name)

p1 = persona("John", 36)
p1.myfunc()
```

Hello my name is John

Modificar propiedades de objeto

```
In [ ]: p1.age = 40
```

Eliminar propiedades de objeto

```
In [ ]: del p1.age
```

Eliminar objetos


```
In [ ]: del p1
```

La declaración de aprobación

```
In [ ]: class Person:
        pass          # para evitar obtener un error, en caso la clase este vacía
```